

Asynchronous coding in C#: issues and troubleshooting

prepared by Anatolii Pastushuk

Introduction

1. How Async Works in C#

- a. Async/await and TAP (Task-based Asynchronous Pattern)
- b. Synchronization Context
- c. Underlying Mechanism

2. Common Issues with Async Programming

- a. Deadlocks
- b. Context Capturing and ConfigureAwait
- c. Exception Handling
- d. Race Conditions and Concurrency
- e. Cancellation and Timeouts

3. Real-World Cases Quiz

Async/await and TAP (Task-based Asynchronous Pattern)

Overview of TAP:

- Standard asynchronous model in .NET
- Uses `Task` and `Task<T>` to represent future results
- Methods typically named with an “Async” suffix (e.g., `GetDataAsync`)

Async/await Keywords:

- **async**: Enables use of **await** and transforms methods into state machines
- **await**: Pauses execution until the awaited task completes (non-blocking)

Key Benefits:

- Improves code readability and maintainability
- Simplifies error handling (exceptions propagate naturally)
- Enhances UI responsiveness and scalability

```
public async Task<string> GetDataAsync()
{
    using (var client = new HttpClient())
    {
        return await client.GetStringAsync("https://example.com");
    }
}
```

Synchronization Context

Definition:

- Abstraction that represents the environment where code executes (e.g., UI thread, ASP.NET request context)
- Ensures that code after an `await` resumes on the proper thread

Why It Matters:

- **UI Applications:** Guarantees that UI updates occur on the main thread
- **Deadlock Prevention:** Improper handling (e.g., blocking calls like `.Result`) can cause deadlocks

Use `ConfigureAwait(false)` in library or non-UI code to avoid unnecessary context capturing

```
public async Task<string> GetDataAsync()
{
    using (var client = new HttpClient())
    {
        return await client.GetStringAsync("https://example.com")
            .ConfigureAwait(false);
    }
}
```

Underlying Mechanism of Async/Await

Compiler-Generated State Machines:

- Async methods are transformed into state machines implementing `IAsyncStateMachine`
- The state machine tracks local state and the current execution point (via `MoveNext()`)

Role of the Task Parallel Library (TPL):

- Manages task scheduling and execution on the thread pool
- Handles continuations based on the captured synchronization context

The Awaiter Pattern:

- `GetAwaiter()` checks if the task is complete or registers a continuation if not
- Ensures that once the task completes, the continuation resumes on the correct thread

Error Handling:

- Exceptions in awaited tasks are captured and rethrown at the await point, streamlining error management

Common Issues with Async Programming

Deadlocks

Definition:

A deadlock occurs when two or more operations are waiting on each other to complete, resulting in an indefinite wait.

Common Scenario:

Calling an async method synchronously (e.g., using `.Result` or `.Wait()`) on the UI thread.

How It Happens:

The async method captures the current synchronization context (e.g., UI thread) and schedules its continuation there. When the thread is blocked, the continuation never runs, resulting in a deadlock.

```
public string GetData()  
{  
    // Blocks the UI thread waiting for the async task to complete.  
    return GetDataAsync().Result;  
}
```

What happens when an async method that captures the synchronization context is called synchronously on the UI thread using `.Result`?

Context Capturing and ConfigureAwait

Context Capturing:

When `await` is used, it captures the current synchronization context (e.g., UI thread) to resume the continuation.

Potential Downsides:

Can cause performance overhead and deadlocks if the context becomes blocked.

Using ConfigureAwait(false):

Instructs the awaiter not to capture the synchronization context, allowing continuations to run on any available thread. Ideal for library code or background operations.

```
public async Task<string> GetDataAsync()
{
    using (var client = new HttpClient())
    {
        return await client.GetStringAsync("https://example.com")
                    .ConfigureAwait(false);
    }
}
```

Why would you use ConfigureAwait(false) in non-UI or library code?

Exception Handling

Task-Based Exception Propagation:

Exceptions thrown in async methods are captured within the Task and rethrown at the await point.

Async Void Pitfall:

async void methods cannot be awaited, meaning exceptions they throw cannot be caught externally.

Aggregate Exceptions:

When multiple tasks are awaited (e.g., with Task.WhenAll), exceptions may be aggregated into an AggregateException.

```
public async Task ProcessDataAsync()
{
    try
    {
        string data = await GetDataAsync();
        // Process data...
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Error: {ex.Message}");
    }
}
```

What is the risk of using async void methods in terms of exception handling?

Race Conditions and Concurrency

Definition:

Race conditions occur when multiple async operations access and modify shared data concurrently, leading to inconsistent or unpredictable results.

Example Scenario:

Concurrently incrementing a shared counter without proper synchronization.

Prevention Strategies:

- Use synchronization primitives like `lock`, `Interlocked`, or asynchronous locks (e.g., `SemaphoreSlim`).
- Employ thread-safe or immutable data structures.

```
private readonly object _lock = new object();

public async Task IncrementCounterWithLockAsync()
{
    for (int i = 0; i < 1000; i++)
    {
        await Task.Delay(1);
        lock (_lock)
        {
            _counter++;
        }
    }
}
```

How can you prevent a race condition when multiple async operations try to increment a shared counter?

Cancellation and Timeouts

Cancellation Tokens:

Allow you to signal that an asynchronous operation should be canceled. Use `CancellationToken` and `CancellationTokenSource` to manage this.

Timeouts:

Prevent operations from running indefinitely by automatically canceling them after a specified period.

Implementation:

Pass the cancellation token to async methods, periodically check for cancellation, and handle `OperationCanceledException`.

```
async Task<string> GetDataWithTimeoutAsync()
{
    using (var client = new HttpClient())
    {
        using (var cts = new CancellationTokenSource(TimeSpan.FromSeconds(5)))
        {
            HttpResponseMessage response = await client.GetAsync("https://example.com", cts.Token);
            cts.Token.ThrowIfCancellationRequested();
            return await response.Content.ReadAsStringAsync();
        }
    }
}
```

Can you automatically cancel an async operation if it exceeds a specified time limit?

Some code examples and quiz

Multithreaded programming

